UNIVERSIDAD NACIONAL DEL ALTIPLANO

Facultad de Ingeniería Mecánica Eléctrica, Electrónica y Sistemas

Escuela Profesional de Ingeniería de Sistemas

Trabajo Encargado N.º 03 · Problemas de concurrencia

El problema del barbero dormilón:

Simulador de hilos, semáforos y exclusión mutua para N barberos, sillas y clientes

Curso: SIS 215 – Sistemas Operativos

Docente: Ing. Donia Ruelas

Integrante: Emerson Aldair Flores Flores

Semestre: 2026-II

Fecha: 12 de octubre de 2026

Repositorio del simulador:

https://github.com/Enerdev/Proyecto-final-Sistem-operative

Puno – Perú, 2026

Índice general

Resumen **1**

1 Introducción **1**

2 Objetivos **2**

2.1 Objetivo general 2

2.2 Objetivos específicos 2

3 Marco teórico **2**

3.1 Procesos e hilos 2

3.2 Condición de carrera y sección crítica 3

3.3 Exclusión mutua y semáforos 3

3.4 Interbloqueo e inanición 3

3.5 El problema del barbero dormilón 4

3.6 Políticas de atención 4

3.7 Privacidad y protección de datos personales 5

4 Solución propuesta **5**

4.1 Variables compartidas y semáforos 5

4.2 Algoritmo 5

4.3 Análisis de corrección 6

4.4 Implementación en C con `pthreads` 7

4.5 Ejecución del programa en C 8

5 Descripción del simulador **8**

5.1 Arquitectura 8

5.2 Pantalla de inicio 9

5.3 Interfaz principal 10

5.4 Configuración y selección de casos 11

5.5 Barbería: barberos y sala de espera 13

5.6 Memoria compartida y estadísticas 13

5.7 Diagrama de Gantt 14

5.8 Hilos cliente y bitácora de eventos 14

5.9 Demostración de la condición de carrera 15
5.10 Casos de prueba y ejecución en lote 16
5.11 Pestañas de código, teoría y políticas 17
5.12 Tema oscuro y diseño adaptable 20

6 Análisis de resultados **21**
6.1 Resultados de los nueve casos 21
6.2 Interpretación 22

7 Políticas de uso y privacidad **23**
7.1 Políticas de uso 23
7.2 Política de privacidad 23

8 Conclusiones **24**

Referencias **24**

Anexo A. Repositorio y ejecución **26**

Anexo B. Código fuente completo **26**

Índice de figuras

1 Salida de la ejecución del programa en C 8
2 Pantalla de inicio del simulador 9
3 Selección del caso inicial y aceptación de políticas 10
4 Vista general del simulador durante el caso clásico 11
5 Panel de configuración 12
6 Barbería grande con tres barberos y cinco sillas 13
7 Panel de memoria compartida 13
8 Franja de estadísticas de la simulación 14
9 Diagrama de Gantt del caso Barbería grande 14
10 Tabla de hilos cliente y bitácora de eventos 15
11 Sala de espera con clientes de pie por la condición de carrera 15
12 Memoria compartida corrompida sin mutex 16
13 Bitácora con la condición de carrera detectada 16
14 Pestaña de casos de prueba 17
15 Comparación en lote de los nueve casos 17
16 Historial de ejecuciones del simulador 17
17 Pestaña de código con la solución en C 18
18 Pestaña de teoría 19
19 Pestaña de políticas de uso y privacidad 19
20 Simulador en tema oscuro 20
21 Caso de prioridad sin envejecimiento en tema oscuro 20
22 Vista en un celular: pantalla de inicio y simulador 21
23 Espera promedio y máxima según la política de atención 22

Índice de tablas

1 Correspondencia entre la barbería y un sistema operativo 4
2 Primitivas de sincronización de la solución 5
3 Resultados de la simulación de los nueve casos de prueba 21

Resumen

El problema del barbero dormilón es un problema clásico de sincronización entre procesos concurrentes. En una barbería, uno o más barberos atienden a clientes que llegan en momentos aleatorios; si no hay clientes, los barberos duermen; si llegan clientes y todos los barberos están ocupados, esperan en un número limitado de sillas; y si la sala está llena, se retiran. Este informe presenta una solución basada en dos semáforos contadores y un mutex, generalizada a N barberos, S sillas y N clientes, implementada con hilos reales en C (POSIX `pthreads`) y en Python (`threading`). Además, se desarrolló un simulador web interactivo que muestra en tiempo real el estado de cada hilo, la memoria compartida, un diagrama de Gantt por barbero y una bitácora de operaciones `wait` y `signal`. El simulador permite comparar tres políticas de atención (FIFO, SJF y prioridad con envejecimiento) y desactivar el mutex para observar una condición de carrera. Se analizaron nueve casos de prueba. Los resultados confirman que la solución evita el interbloqueo, que la política FIFO evita la inanición y que, sin exclusión mutua, el contador compartido se corrompe y la barbería admite más clientes de los que caben. El informe incluye también las políticas de uso y privacidad del simulador, alineadas con la Ley N.º 29733.

Palabras clave: concurrencia, hilos, semáforos, exclusión mutua, condición de carrera, inanición, barbero dormilón.

1. Introducción

Un sistema operativo moderno ejecuta muchos flujos de ejecución a la vez. Los hilos de un mismo proceso comparten memoria, lo que hace la comunicación entre ellos rápida, pero también peligrosa: si dos hilos modifican el mismo dato sin coordinación, el resultado depende del orden en que el planificador los intercale (Ruelas, 2026). Para estudiar estos riesgos, la literatura de sistemas operativos usa un conjunto de «problemas clásicos»: productor–consumidor, lectores–escritores, filósofos comensales, fumadores y barbero dormilón (Maziero, 2019; Wolf et al., 2015).

Al equipo le correspondió el problema del barbero dormilón. Su interés está en que combina tres situaciones típicas de un sistema operativo real: un servidor que debe *dormir* cuando no hay trabajo para no gastar CPU, una *cola de capacidad limitada* que rechaza peticiones cuando se llena, y un *contador compartido* que todos los hilos leen y modifican. La barbería es, en la práctica, un modelo de un servidor web con un *pool* de hilos trabajadores y una cola de conexiones pendientes.

El informe se organiza así: la sección 2 plantea los objetivos; la sección 3 resume los conceptos teóricos; la sección 4 presenta el algoritmo y su implementación; la sección 5 describe el simulador con capturas de cada módulo; la sección 6 analiza los resultados de

nueve casos; la sección 7 expone las políticas de uso y privacidad; y la sección 8 presenta las conclusiones.

2. Objetivos

2.1 Objetivo general

Resolver el problema del barbero dormilón mediante hilos y primitivas de sincronización, y construir un simulador interactivo que permita observar y analizar su comportamiento para cualquier número de barberos, sillas y clientes.

2.2 Objetivos específicos

1. Modelar barberos y clientes como hilos de un mismo proceso que comparten la sala de espera.
2. Diseñar una solución con semáforos y mutex que garantice exclusión mutua y evite el interbloqueo.
3. Implementar la solución con hilos reales en C (`pthreads`) y en Python (`threading`).
4. Desarrollar un simulador web que muestre el estado de los hilos, la memoria compartida, un diagrama de Gantt y una bitácora de eventos.
5. Comparar las políticas de atención FIFO, SJF y prioridad, y evaluar el efecto del envejecimiento sobre la inanición.
6. Demostrar experimentalmente la condición de carrera que aparece al eliminar la exclusión mutua.
7. Definir las políticas de uso y privacidad del simulador conforme a la legislación peruana de protección de datos personales.

3. Marco teórico

3.1 Procesos e hilos

Un proceso es una instancia de un programa en ejecución, con su propio espacio de memoria y recursos. Un hilo (*thread*) es una unidad de ejecución dentro de un proceso que tiene menos contexto propio que un proceso completo: cada hilo tiene su propia pila, contador de programa y registros, pero comparte con los demás hilos el código, los datos globales y los archivos abiertos (Ruelas, 2026; Mandl, 2020). El sistema operativo planifica los hilos, no los procesos, y puede existir un número de hilos mucho mayor que el de CPU lógicas disponibles.

En el problema estudiado, la barbería es un proceso; cada barbero y cada cliente es un hilo. Los datos de cada cliente (su identificador, la duración de su corte y su prioridad) viven en su pila y son *privados*. La sala de espera y el contador de clientes esperando son *compartidos*.

3.2 Condición de carrera y sección crítica

Una condición de carrera ocurre cuando dos o más hilos acceden y modifican un mismo dato compartido, y el resultado depende del orden en que se ejecutan sus instrucciones (Ruelas, 2026). Por ejemplo, la instrucción `esperando++` se traduce a tres operaciones de máquina: leer, sumar y escribir. Si dos clientes la ejecutan de forma intercalada, ambos leen el mismo valor y uno de los incrementos se pierde.

La sección crítica es la parte del código que accede al recurso compartido y que debe ejecutarse de manera controlada. Maziero (2019) establece que una solución correcta debe cumplir exclusión mutua, progreso y espera limitada.

3.3 Exclusión mutua y semáforos

La exclusión mutua garantiza que solo un hilo use un recurso compartido en un instante dado. Los mecanismos más habituales son mutex, *locks*, semáforos y monitores (Ruelas, 2026; Hoare, 1974). Un semáforo es una variable entera con dos operaciones atómicas:

- `wait(s)` (también `down` o `P`): si $s > 0$, lo decrementa; si $s = 0$, bloquea al hilo hasta que otro haga `signal`.
- `signal(s)` (también `up` o `V`): incrementa s y, si hay hilos bloqueados, despierta a uno.

Un mutex es un semáforo binario que inicia en 1. Hoare (1974) propuso los monitores como una forma estructurada de agrupar los datos compartidos con los procedimientos que los protegen, y más tarde formalizó la comunicación entre procesos secuenciales mediante paso de mensajes (Hoare, 1978).

3.4 Interbloqueo e inanición

Hay interbloqueo (*deadlock*) cuando dos o más hilos quedan bloqueados indefinidamente porque cada uno espera un recurso que retiene otro. Para que ocurra deben cumplirse a la vez cuatro condiciones: exclusión mutua, retención y espera, no expropiación y espera circular (Mandl, 2020; Wolf et al., 2015). Romper cualquiera de ellas lo evita.

La inanición (*starvation*) ocurre cuando un hilo espera indefinidamente porque otros obtienen siempre la prioridad (Ruelas, 2026). Es un riesgo de las políticas que no

atienden por orden de llegada, como SJF o prioridad estática. El *envejecimiento* (*aging*) la corrige aumentando la prioridad de un hilo conforme pasa el tiempo que lleva esperando (Maziero, 2019).

3.5 El problema del barbero dormilón

Una barbería tiene B barberos con sus sillas de corte y una sala de espera con S sillas. Las reglas son:

1. Si no hay clientes, el barbero se duerme en su silla.
2. Cuando llega un cliente y hay un barbero dormido, lo despierta.
3. Si todos los barberos están ocupados y hay sillas libres, el cliente se sienta a esperar.
4. Si la sala está llena, el cliente se retira sin ser atendido.

El problema exige que ningún barbero duerma mientras hay clientes esperando, que ningún cliente espere sin que un barbero vaya a atenderlo, y que el número de clientes en la sala nunca supere S.

Tabla 1
Correspondencia entre la barbería y un sistema operativo

Elemento de la barbería	**Equivalente en el sistema operativo**
Barbería	Proceso que contiene a todos los hilos
Barbero	Hilo trabajador de un *pool* (servidor)
Cliente	Hilo o petición que solicita un servicio
Sala de espera con S sillas	Búfer o cola acotada de peticiones pendientes
Barbero dormido	Hilo bloqueado en un semáforo, sin consumir CPU
Cliente que se va	Petición rechazada por cola llena
Contador `esperando`	Variable compartida en la sección crítica
Elegir al siguiente cliente	Política de planificación (FIFO, SJF, prioridad)

Nota. Elaboración propia.

3.6 Políticas de atención

Cuando un barbero se libera y hay varios clientes esperando, debe elegir a cuál atender. Es el mismo problema que resuelve el planificador de CPU (Maziero, 2019; Mandl, 2020):

- **FIFO** (*First In, First Out*): atiende por orden de llegada. Es justa y no produce inanición.

- **SJF** (*Shortest Job First*): atiende primero el corte más corto. Minimiza la espera promedio, pero un corte largo puede ser postergado indefinidamente.
- **Prioridad**: atiende primero al cliente de mayor prioridad (1 = alta). Sin envejecimiento produce inanición en los clientes de prioridad baja.

3.7 Privacidad y protección de datos personales

En el plano técnico, el sistema operativo aísla los datos privados de cada hilo (su pila y registros) y solo permite modificar los datos compartidos dentro de una sección crítica. En el plano legal, cualquier software que registre datos de personas en el Perú está sujeto a la Ley N.º 29733, Ley de Protección de Datos Personales (Congreso de la República del Perú, 2011), y a su reglamento, aprobado por el Decreto Supremo N.º 016-2024-JUS (Ministerio de Justicia y Derechos Humanos, 2024). Estas normas exigen consentimiento previo e informado, uso de los datos solo para la finalidad declarada, medidas de seguridad y el respeto de los derechos de acceso, rectificación, cancelación y oposición (ARCO).

4. Solución propuesta

4.1 Variables compartidas y semáforos

La solución usa tres primitivas de sincronización y una variable compartida (Tabla 2).

Tabla 2
Primitivas de sincronización de la solución

Nombre	Valor inicial	Función
`clientes`	0	Semáforo contador: número de clientes esperando. El barbero hace `wait(clientes)` y duerme si vale 0.
`barberos`	0	Semáforo contador: barberos listos para atender. El cliente hace `wait(barberos)` hasta que lo llamen.
`mutex`	1	Semáforo binario: garantiza la exclusión mutua sobre `esperando`.
`esperando`	0	Variable entera compartida: clientes sentados en la sala.

4.2 Algoritmo

El Listado 1 muestra el algoritmo generalizado a B barberos. Se crean B hilos barbero, que se repiten indefinidamente, y N hilos cliente, que se ejecutan una vez.

Listado 1
Pseudocódigo de la solución para B barberos

```
semáforo clientes = 0       // clientes esperando
semáforo barberos = 0       // barberos listos para cortar
semáforo mutex    = 1       // exclusión mutua sobre 'esperando'
entero   esperando = 0
constante SILLAS = S

hilo Barbero(id):                   // se crean B hilos
    repetir siempre:
        wait(clientes)              // si no hay clientes, se duerme
        wait(mutex)
            esperando = esperando - 1
            signal(barberos)        // un barbero listo
        signal(mutex)
        cortar_pelo()

hilo Cliente(id):                   // se crean N hilos
    wait(mutex)
    si esperando < SILLAS:
        esperando = esperando + 1
        signal(clientes)            // despierta a un barbero si duerme
        signal(mutex)
        wait(barberos)              // espera a que lo llamen
        recibir_corte()
    si no:
        signal(mutex)               // sala llena: se va
```

4.3 Análisis de corrección

Exclusión mutua

Toda lectura y escritura de `esperando` ocurre entre `wait(mutex)` y `signal(mutex)`. Como `mutex` inicia en 1, a lo sumo un hilo está dentro de la sección crítica, por lo que la comparación `esperando < SILLAS` y el incremento posterior son atómicos respecto de los demás hilos.

Ausencia de interbloqueo

Ningún hilo retiene el mutex mientras espera otro semáforo: el cliente ejecuta `signal(mutex)` *antes* de `wait(barberos)`, y el barbero libera el mutex antes de cortar. Así se rompe la condición de *retención y espera* y no puede formarse una espera circular (Mandl, 2020).

Barbero dormido sin desperdiciar CPU

El barbero se bloquea en `wait(clientes)`, no en un bucle de espera activa. El sistema operativo lo saca de la cola de listos hasta que un cliente ejecute `signal(clientes)`. Cada `signal(clientes)` corresponde a exactamente un cliente sentado, por lo que nunca se despierta a un barbero sin trabajo.

Inanición

Con semáforos FIFO, la solución atiende en orden de llegada y ningún cliente admitido espera indefinidamente. Si se cambia la política de selección (SJF o prioridad), la inanición vuelve a ser posible; el simulador la mide y la corrige con envejecimiento (sección 6).

4.4 Implementación en C con `pthreads`

El Listado 2 muestra las funciones de los hilos. El programa completo está en el Anexo B. Se compila con `gcc barbero.c -o barbero -pthread`.

Listado 2

Hilos barbero y cliente en C (fragmento)

```
-
void *barbero(void *arg) {
    int id = *(int *)arg;                          /* dato privado (pila) */
    while (1) {
        sem_wait(&clientes);                       /* duerme si no hay
            clientes */
        pthread_mutex_lock(&mutex);                /* --- sección crítica
            --- */
        esperando--;
        sem_post(&barberos);                       /* llama a un cliente */
        printf("[B%d] corta el pelo   | esperando = %d\n", id, esperando);
        pthread_mutex_unlock(&mutex);              /* --- fin sección crí
            tica --- */
        usleep((200 + rand() % 300) * 1000);       /* cortar_pelo() */
    }
    return NULL;
}

void *cliente(void *arg) {
    int id = *(int *)arg;
    pthread_mutex_lock(&mutex);                    /* --- sección crítica
        --- */
    if (esperando < SILLAS) {
        esperando++;
        printf("  C%-2d se sienta       | esperando = %d\n", id, esperando);
        sem_post(&clientes);                       /* despierta a un barbero
            */
        pthread_mutex_unlock(&mutex);              /* liberar ANTES de
            esperar */
        sem_wait(&barberos);                       /* espera su turno */
        pthread_mutex_lock(&mutex); atendidos++; pthread_mutex_unlock(&mutex
            );
    } else {
        rechazados++;
        printf("  C%-2d se va: sala llena\n", id);
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

4.5 Ejecución del programa en C

La Figura 1 muestra la salida real del programa con 2 barberos, 4 sillas y 15 clientes. Se observa cómo los clientes se sientan, cómo los barberos los toman reduciendo `esperando`, y el resumen final.

Figura 1
Salida de la ejecución del programa en C

```
$ gcc barbero.c -o barbero -pthread && ./barbero
C1  se sienta       | esperando = 1
[B2] corta el pelo   | esperando = 0
C2  se sienta       | esperando = 1
[B1] corta el pelo   | esperando = 0
C3  se sienta       | esperando = 1
[B2] corta el pelo   | esperando = 0
C4  se sienta       | esperando = 1
C5  se sienta       | esperando = 2
C6  se sienta       | esperando = 3
[B1] corta el pelo   | esperando = 2
⋮
[B2] corta el pelo   | esperando = 1
[B2] corta el pelo   | esperando = 0
Atendidos: 13 | Se fueron: 2
```

Nota. Ejecución en Linux con GCC. Por la naturaleza concurrente, el orden de las líneas cambia entre ejecuciones; el total de atendidos más rechazados siempre suma 15.

5. Descripción del simulador

5.1 Arquitectura

El simulador es una aplicación web de un solo archivo (HTML, CSS y JavaScript), sin dependencias externas y sin servidor. Se basa en un motor de *simulación por tiempo discreto*: el reloj avanza en unidades (u) y, en cada unidad, el motor ejecuta cuatro fases en orden fijo:

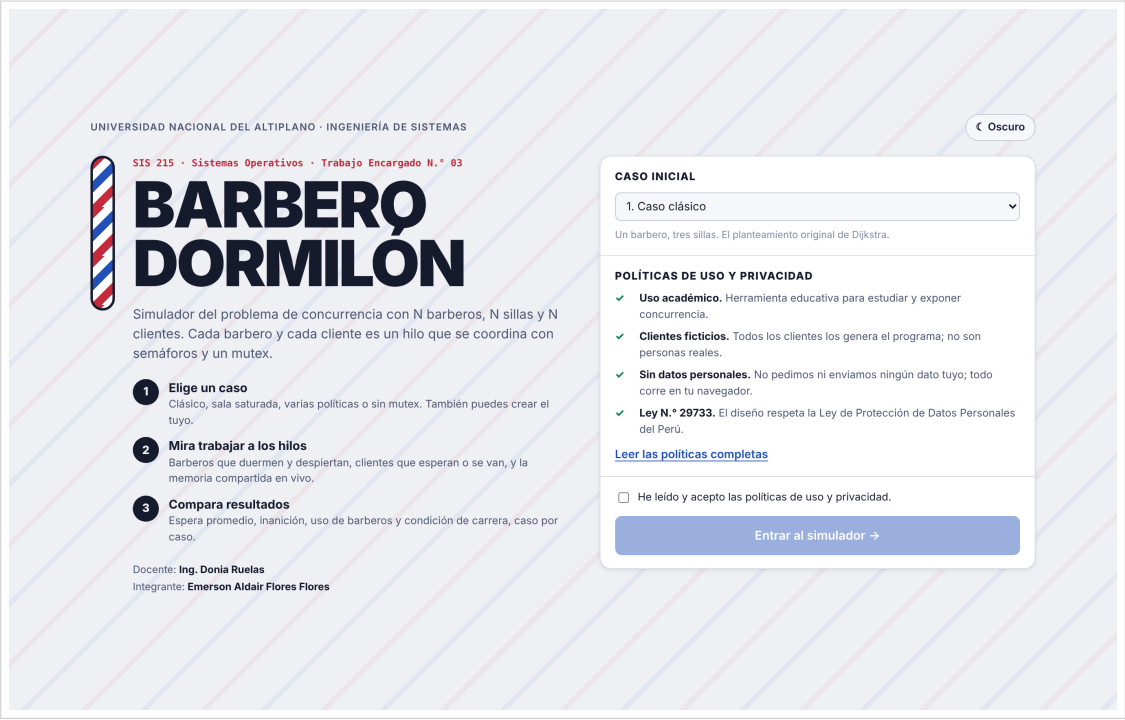
1. Los barberos que terminan su corte liberan su silla; el hilo cliente finaliza.
2. Los barberos libres ejecutan `wait(clientes)` y toman al siguiente cliente según la política elegida, o se duermen si no hay nadie.
3. Llegan los nuevos hilos cliente: ejecutan `wait(mutex)`, comparan `esperando` con el número de sillas, y se sientan o se van.
4. Los barberos que siguen dormidos despiertan por los recién llegados.

Las llegadas, duraciones y prioridades se generan con un generador pseudoaleatorio con semilla (*mulberry32*), de modo que cada caso es reproducible. Cuando el mutex está desactivado, todos los hilos de una misma unidad leen el valor de `esperando` del inicio de la unidad y el último en escribir sobrescribe a los demás, que es exactamente una actualización perdida.

5.2 Pantalla de inicio

Al abrir el simulador se muestra una pantalla de inicio que evita sobrecargar la interfaz principal (Figura 2). Presenta el problema, los tres pasos de uso, el caso inicial y un resumen de las políticas de uso y privacidad. El botón «Entrar al simulador» solo se habilita tras aceptar las políticas (Figura 3).

Figura 2
Pantalla de inicio del simulador



Nota. La pantalla incluye el poste de barbero animado, la descripción del problema y los datos del curso.

Figura 3

Selección del caso inicial y aceptación de políticas



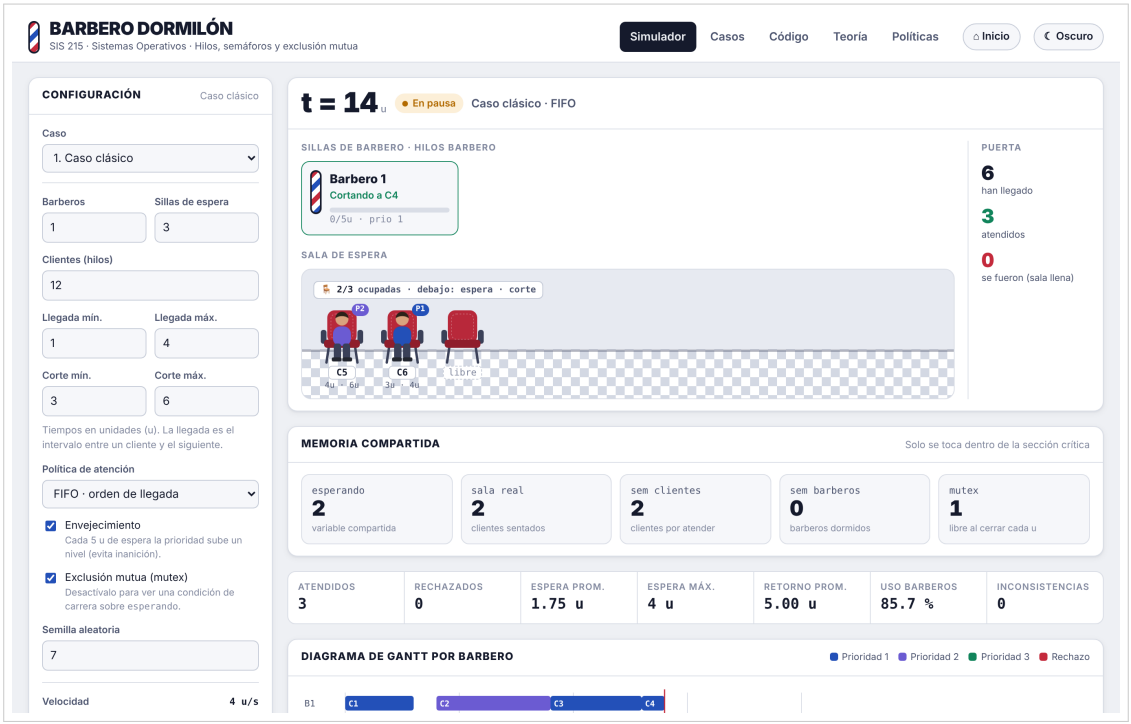
Nota. El enlace «Leer las políticas completas» despliega el texto íntegro sin salir de la pantalla.

5.3 Interfaz principal

La interfaz se divide en un panel de configuración a la izquierda y la barbería en vivo a la derecha (Figura 4). En la parte superior hay cinco pestañas: Simulador, Casos, Código, Teoría y Políticas, además de los botones de Inicio y de cambio de tema.

Figura 4

Vista general del simulador durante el caso clásico



Nota. Instante $t = 14$ u: el barbero corta a C4 mientras C5 y C6 esperan en la sala.

5.4 Configuración y selección de casos

El panel de configuración (Figura 5) permite elegir uno de los casos predefinidos o definir uno propio con cualquier número de barberos (1–12), sillas (0–30) y clientes (1–300), los intervalos de llegada y de corte, la política de atención, el envejecimiento, el mutex y la semilla. Los botones permiten iniciar o pausar, avanzar una unidad a la vez, reiniciar, ejecutar hasta el final y guardar la configuración como un caso nuevo.

Figura 5
Panel de configuración



Nota. Al modificar cualquier parámetro, el selector cambia a «Personalizado» y la simulación se reinicia.

5.5 Barbería: barberos y sala de espera

Cada barbero se representa con una tarjeta que indica su estado (Figura 6): *dormido* (en violeta, bloqueado en `wait(clientes)`), *cortando* (en verde, con barra de progreso y poste girando) o *libre*. La sala de espera dibuja una silla por cada lugar disponible. Un cliente sentado se muestra con el color de su prioridad, su identificador, el tiempo que lleva esperando y la duración de su corte. La sección «Puerta» cuenta los clientes que llegaron, los atendidos y los que se fueron.

Figura 6
Barbería grande con tres barberos y cinco sillas



Nota. Caso 3 en $t = 16$ u: los tres hilos barbero están cortando y la sala está casi llena.

5.6 Memoria compartida y estadísticas

El panel de memoria compartida (Figura 7) muestra el valor de la variable `esperando`, el número real de clientes en la sala, los semáforos `clientes` y `barberos` y el estado del mutex. Si `esperando` deja de coincidir con la sala real, la tarjeta se marca en rojo. Debajo, la franja de estadísticas (Figura 8) resume atendidos, rechazados, espera promedio y máxima, tiempo de retorno promedio, uso de barberos e inconsistencias.

Figura 7
Panel de memoria compartida



Nota. Con mutex activo, `esperando` siempre coincide con la sala real.

Figura 8
Franja de estadísticas de la simulación

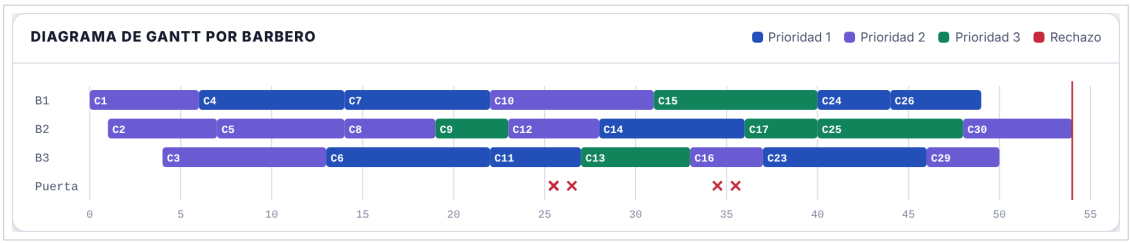


Nota. Espera = inicio del corte − llegada. Uso de barberos = tiempo cortando / ($B\times$ tiempo total).

5.7 Diagrama de Gantt

El diagrama de Gantt (Figura 9) dibuja una fila por barbero con los cortes realizados, coloreados según la prioridad del cliente, y una fila «Puerta» donde una cruz roja marca cada cliente rechazado. Los huecos en una fila corresponden a intervalos en que el barbero estuvo dormido.

Figura 9
Diagrama de Gantt del caso Barbería grande



Nota. Simulación completa: 23 clientes atendidos y 7 rechazados en 54 u.

5.8 Hilos cliente y bitácora de eventos

La tabla de hilos cliente (Figura 10) muestra los datos privados de cada hilo: instante de llegada, duración del corte, prioridad, estado (*nuevo*, *esperando*, *cortando*, *atendido* o *rechazado*), barbero asignado y espera. La bitácora registra cada operación con su instante, incluidas las llamadas `wait` y `signal`, lo que permite seguir la ejecución paso a paso durante la exposición.

Figura 10

Tabla de hilos cliente y bitácora de eventos



Nota. Cada línea de la bitácora tiene un color según el tipo de evento: llegada, corte, sueño, rechazo o fin.

5.9 Demostración de la condición de carrera

Al desactivar el mutex (caso 9, «Sin mutex»), dos clientes que llegan en la misma unidad leen el mismo valor de `esperando` y escriben el mismo resultado, de modo que se pierde una actualización. El contador queda por debajo del número real de clientes, la barbería admite más clientes de los que caben y estos aparecen *de pie* en la sala (Figura 11). El panel de memoria compartida marca la inconsistencia (Figura 12) y la bitácora la registra (Figura 13).

Figura 11

Sala de espera con clientes de pie por la condición de carrera



Nota. Solo hay 3 sillas, pero entraron 5 clientes porque el contador perdió actualizaciones.

Figura 12

Memoria compartida corrompida sin mutex



Nota. La variable `esperando` vale 2 mientras la sala real tiene 5 clientes.

Figura 13

Bitácora con la condición de carrera detectada



HILOS CLIENTE — Datos privados de cada hilo (su pila)

HILO	LLEGA	CORTE	PRIO.	ESTADO	BARBERO	ESPERA
C1	0	3u	3	atendido	B1	0u
C2	0	6u	3	atendido	B2	0u
C3	1	4u	3	atendido	B1	2u
C4	1	5u	3	atendido	B2	5u
C5	2	4u	3	atendido	B1	5u
C6	2	5u	2	cortando	B1	9u
C7	2	5u	3	cortando	B2	9u
C8	3	4u	3	esperando	—	9u
C9	3	5u	2	esperando	—	9u
C10	3	6u	1	esperando	—	9u
C11	4	4u	1	esperando	—	8u

Nota. La última línea, resaltada en rojo, señala la diferencia entre el contador y la sala real.

Lección de la demostración

Una sola instrucción de alto nivel como `esperando++` no es atómica. Sin exclusión mutua, el error no produce un fallo inmediato: el programa sigue funcionando, pero con un estado interno falso que lleva a decisiones equivocadas, como admitir clientes sin sillas o rechazar clientes habiendo sitio.

5.10 Casos de prueba y ejecución en lote

La pestaña Casos (Figura 14) reúne los nueve casos predefinidos y los que el usuario guarde. Cada uno puede cargarse en el simulador o ejecutarse completo en lote para compararlos en una sola tabla (Figura 15). Además, cada simulación terminada se agrega al historial de ejecuciones (Figura 16).

Figura 14

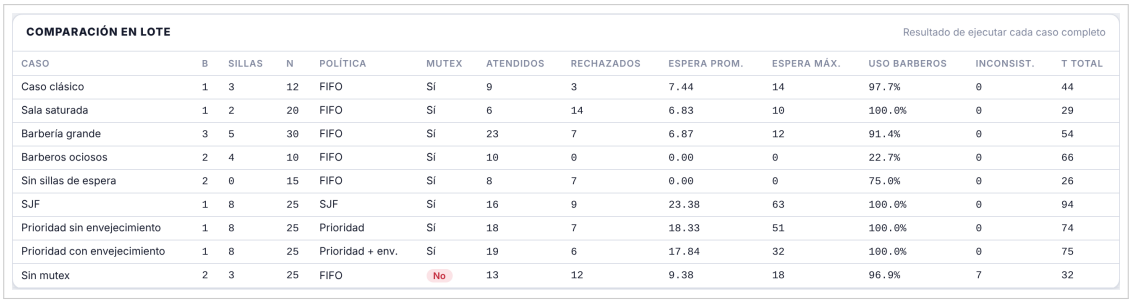
Pestaña de casos de prueba



Nota. Cada tarjeta indica los parámetros del caso: barberos, sillas, clientes, intervalos, política y mutex.

Figura 15

Comparación en lote de los nueve casos



COMPARACIÓN EN LOTE — Resultado de ejecutar cada caso completo

CASO	B	SILLAS	N	POLÍTICA	MUTEX	ATENDIDOS	RECHAZADOS	ESPERA PROM.	ESPERA MÁX.	USO BARBEROS	INCONSIST.	T TOTAL
Caso clásico	1	3	12	FIFO	Sí	9	3	7.44	14	97.7%	0	44
Sala saturada	1	2	20	FIFO	Sí	6	14	6.83	10	100.0%	0	29
Barbería grande	3	5	30	FIFO	Sí	23	7	6.87	12	91.4%	0	54
Barberos ociosos	2	4	10	FIFO	Sí	10	0	0.00	0	22.7%	0	66
Sin sillas de espera	2	0	15	FIFO	Sí	8	7	0.00	0	75.0%	0	26
SJF	1	8	25	SJF	Sí	16	9	23.38	63	100.0%	0	94
Prioridad sin envejecimiento	1	8	25	Prioridad	Sí	18	7	18.33	51	100.0%	0	74
Prioridad con envejecimiento	1	8	25	Prioridad + env.	Sí	19	6	17.84	32	100.0%	0	75
Sin mutex	2	3	25	FIFO	No	13	12	9.38	18	96.9%	7	32

Nota. Resultados obtenidos al ejecutar cada caso hasta el final con su semilla.

Figura 16

Historial de ejecuciones del simulador



HISTORIAL DE EJECUCIONES — Limpiar

#	CASO	B	SILLAS	N	POLÍTICA	MUTEX	ATENDIDOS	RECHAZADOS	ESPERA PROM.	ESPERA MÁX.	USO BARBEROS	INCONSIST.	T TOTAL
1	Caso clásico	1	3	12	FIFO	Sí	9	3	7.44	14	97.7%	0	44
2	Sala saturada	1	2	20	FIFO	Sí	6	14	6.83	10	100.0%	0	29
3	Barbería grande	3	5	30	FIFO	Sí	23	7	6.87	12	91.4%	0	54
4	SJF	1	8	25	SJF	Sí	16	9	23.38	63	100.0%	0	94
5	Prioridad sin envejecimiento	1	8	25	Prioridad	Sí	18	7	18.33	51	100.0%	0	74
6	Prioridad con envejecimiento	1	8	25	Prioridad + env.	Sí	19	6	17.84	32	100.0%	0	75
7	Sin mutex	2	3	25	FIFO	No	13	12	9.38	18	96.9%	7	32

Nota. El historial conserva cada simulación terminada durante la sesión.

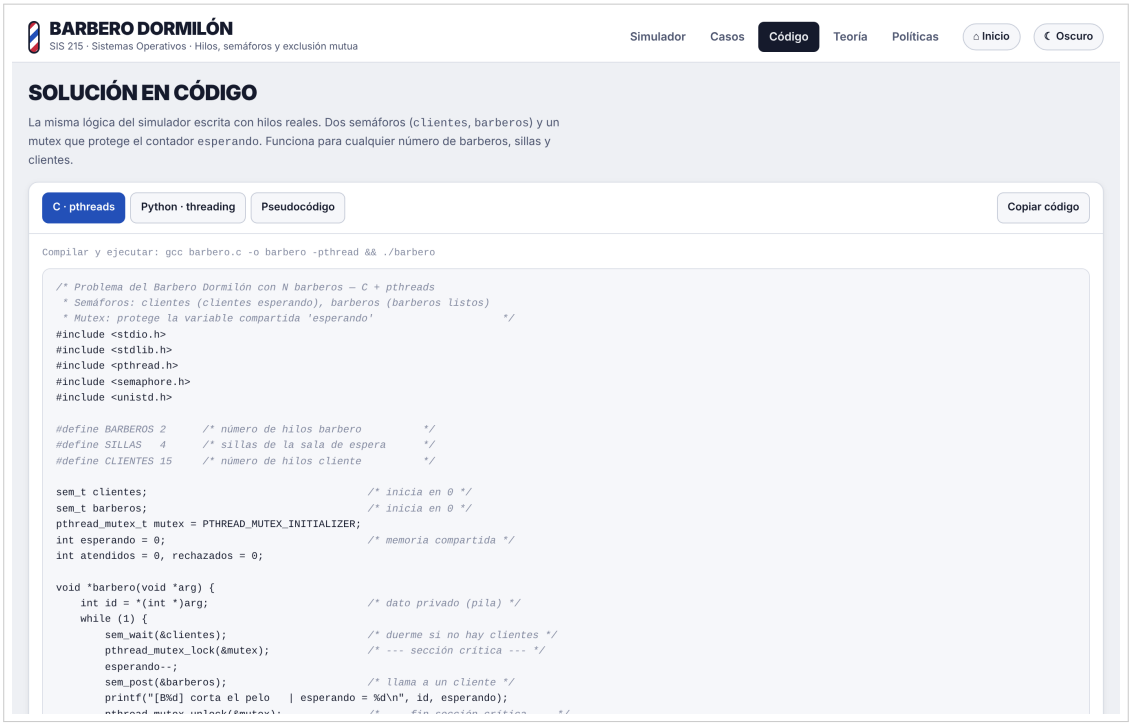
5.11 Pestañas de código, teoría y políticas

La pestaña Código (Figura 17) contiene la solución completa en C, Python y pseudocódigo, con un botón para copiarla. La pestaña Teoría (Figura 18) resume los

conceptos del curso aplicados al problema. La pestaña Políticas (Figura 19) contiene el texto completo de las políticas de uso y privacidad.

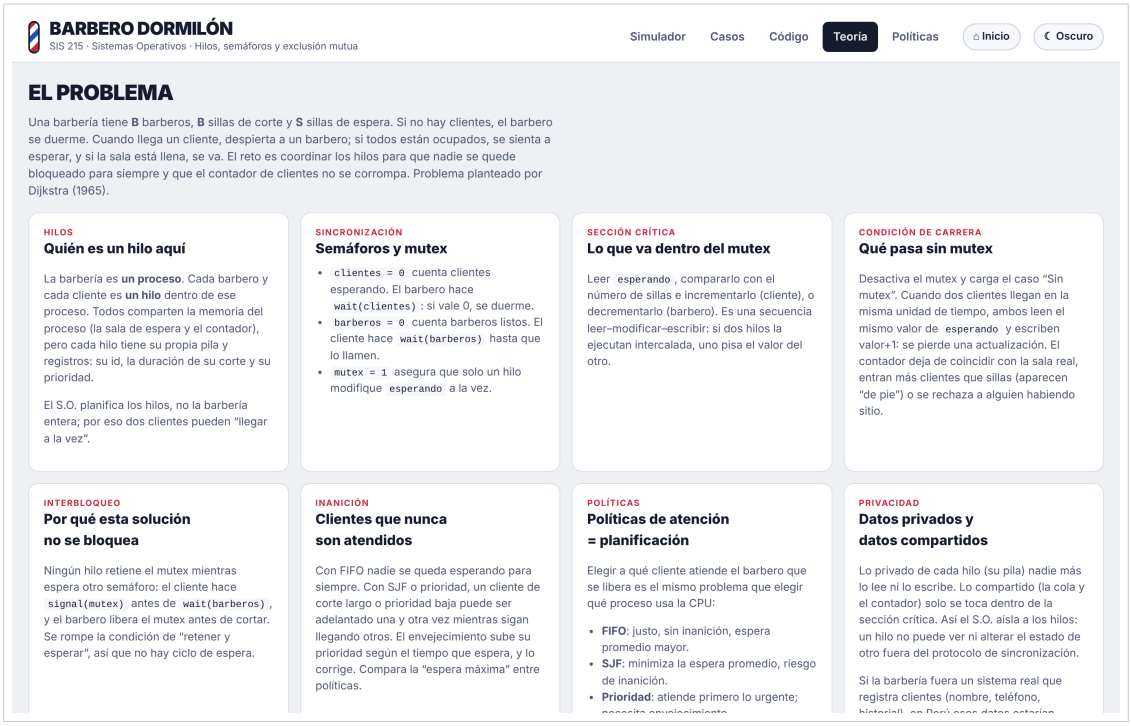
Figura 17

Pestaña de código con la solución en C



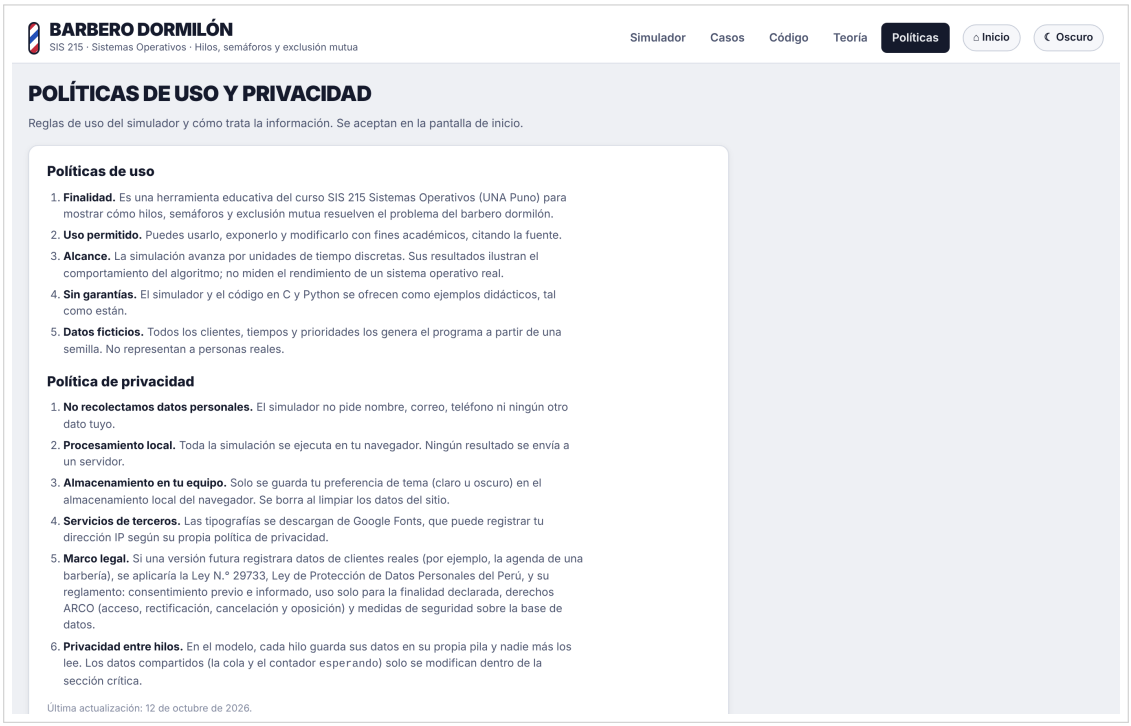
Nota. También están disponibles las versiones en Python y en pseudocódigo.

Figura 18
Pestaña de teoría



Nota. Las tarjetas explican hilos, semáforos, sección crítica, condición de carrera, interbloqueo, inanición, políticas y privacidad.

Figura 19
Pestaña de políticas de uso y privacidad



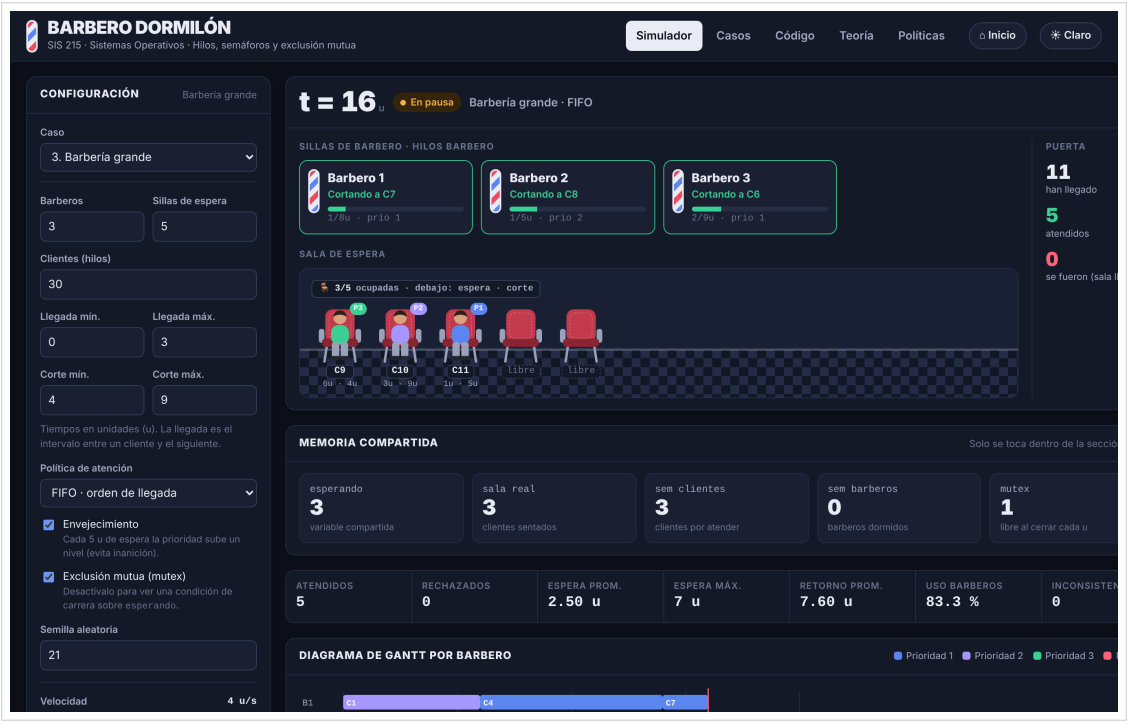
Nota. El contenido se detalla en la sección 7 de este informe.

5.12 Tema oscuro y diseño adaptable

El simulador tiene tema claro y oscuro (Figura 20 y Figura 21). Por defecto sigue la preferencia del sistema operativo, y el botón superior permite cambiarlo. La interfaz se adapta a pantallas de celular (Figura 22).

Figura 20

Simulador en tema oscuro



Nota. Caso Barbería grande en ejecución.

Figura 21

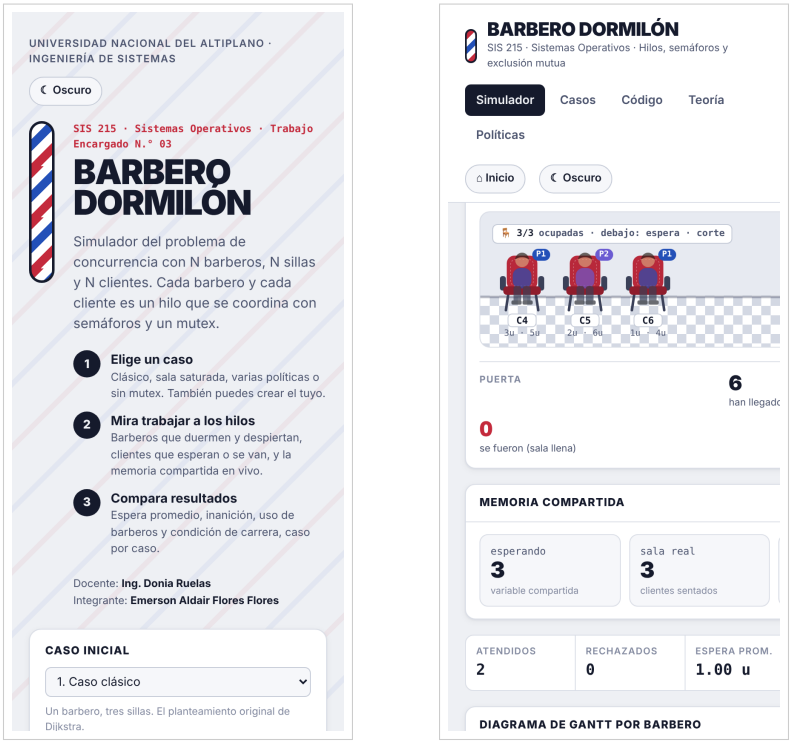
Caso de prioridad sin envejecimiento en tema oscuro



Nota. El icono de reloj de arena marca a los clientes que llevan 10 u o más esperando, señal de posible inanición.

Figura 22

Vista en un celular: pantalla de inicio y simulador



Nota. Ancho de 390 píxeles. Los paneles se apilan en una sola columna.

6. Análisis de resultados

6.1 Resultados de los nueve casos

La Tabla 3 resume los resultados del simulador al ejecutar cada caso completo.

Tabla 3

Resultados de la simulación de los nueve casos de prueba

N.º	Caso	B	S	N	Política	Atend.	Rech.	Esp. prom.	Esp. máx.	Uso
1	Caso clásico	1	3	12	FIFO	9	3	7.44	14	97.7 %
2	Sala saturada	1	2	20	FIFO	6	14	6.83	10	100.0 %
3	Barbería grande	3	5	30	FIFO	23	7	6.87	12	91.4 %
4	Barberos ociosos	2	4	10	FIFO	10	0	0.00	0	22.7 %
5	Sin sillas de espera	2	0	15	FIFO	8	7	0.00	0	75.0 %
6	SJF	1	8	25	SJF	16	9	23.38	63	100.0 %
7	Prioridad sin envejecimiento	1	8	25	Prioridad	18	7	18.33	51	100.0 %
8	Prioridad con envejecimiento	1	8	25	Prioridad + env.	19	6	17.84	32	100.0 %
9	Sin mutex	2	3	25	FIFO (sin mutex)	13	12	9.38	18	96.9 %

Nota. B = barberos; S = sillas de espera; N = clientes. Esperas en unidades de tiempo (u). Uso = porcentaje del tiempo que los barberos pasaron cortando. El caso 9 registró 7 inconsistencias del contador compartido; los demás, ninguna.

6.2 Interpretación

Capacidad y rechazos. En el caso clásico, un barbero con 3 sillas atiende a 9 de 12 clientes con un uso del 97.7 %: el barbero casi no duerme y la sala limitada obliga a rechazar a 3 clientes. Cuando la llegada es más rápida que el servicio (caso 2, «Sala saturada»), se rechaza al 70 % de los clientes y el barbero trabaja el 100 % del tiempo. Agregar barberos y sillas (caso 3) eleva la proporción de atendidos al 77 %.

Barberos ociosos. Con llegadas espaciadas (caso 4), todos los clientes son atendidos sin esperar, pero el uso de los barberos cae al 22.7 %: pasan la mayor parte del tiempo dormidos en `wait(clientes)`, sin consumir CPU. Esto muestra la ventaja de bloquear a un hilo en lugar de usar espera activa.

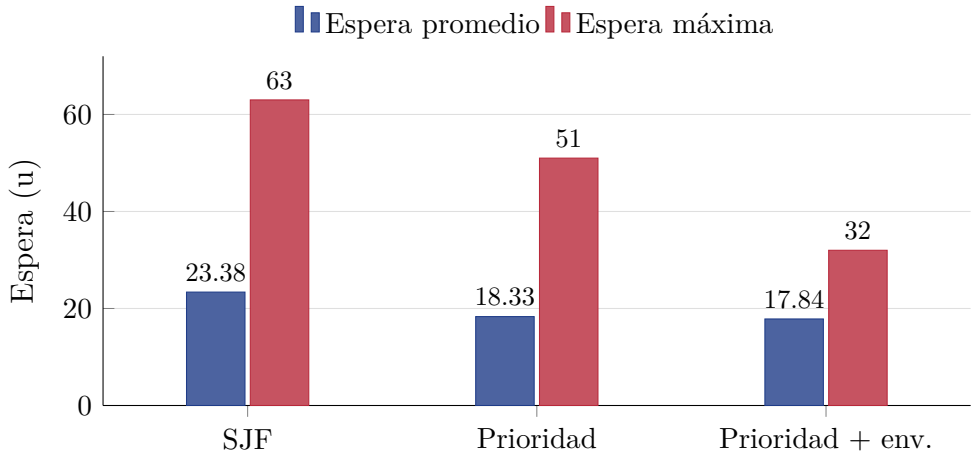
Sin sillas de espera. Con $S = 0$ (caso 5), solo entra quien encuentra un barbero libre; la espera es cero, pero se rechaza casi a la mitad de los clientes.

Políticas e inanición. Con SJF (caso 6), un cliente llegó a esperar 63 u, casi el doble que con prioridad y envejecimiento: los cortes largos fueron adelantados una y otra vez por cortes cortos. Con prioridad estática (caso 7), la espera máxima fue de 51 u, sufrida por clientes de prioridad 3. Con la misma carga y envejecimiento (caso 8), la espera máxima bajó a 32 u (37 % menos) y la espera promedio también disminuyó, porque ningún cliente queda relegado indefinidamente.

Condición de carrera. En el caso 9, sin mutex, el contador `esperando` se desincronizó 7 veces de la sala real. Las consecuencias fueron visibles: entraron clientes que no tenían silla y se rechazó a otros habiendo sitio. Con el mutex activo, los otros ocho casos registraron cero inconsistencias.

Figura 23

Espera promedio y máxima según la política de atención



Nota. Casos 6, 7 y 8 del simulador, con 1 barbero, 8 sillas y 25 clientes.

7. Políticas de uso y privacidad

El simulador incluye políticas de uso y privacidad que el usuario debe aceptar en la pantalla de inicio (Figura 3) y que puede consultar en todo momento en la pestaña Políticas (Figura 19).

7.1 Políticas de uso

1. **Finalidad.** Es una herramienta educativa del curso SIS 215 Sistemas Operativos de la UNA Puno, para mostrar cómo hilos, semáforos y exclusión mutua resuelven el problema del barbero dormilón.
2. **Uso permitido.** Puede usarse, exponerse y modificarse con fines académicos, citando la fuente.
3. **Alcance.** La simulación avanza por unidades de tiempo discretas. Sus resultados ilustran el comportamiento del algoritmo y no miden el rendimiento de un sistema operativo real.
4. **Sin garantías.** El simulador y el código en C y Python se ofrecen como ejemplos didácticos, tal como están.
5. **Datos ficticios.** Todos los clientes, tiempos y prioridades los genera el programa a partir de una semilla y no representan a personas reales.

7.2 Política de privacidad

1. **No se recolectan datos personales.** El simulador no pide nombre, correo, teléfono ni ningún otro dato del usuario.
2. **Procesamiento local.** Toda la simulación se ejecuta en el navegador del usuario y ningún resultado se envía a un servidor.
3. **Almacenamiento en el equipo.** Solo se guarda la preferencia de tema (claro u oscuro) en el almacenamiento local del navegador, y se borra al limpiar los datos del sitio.
4. **Servicios de terceros.** Las tipografías se descargan de Google Fonts, que puede registrar la dirección IP del usuario según su propia política de privacidad.
5. **Marco legal.** Si una versión futura registrara datos de clientes reales, por ejemplo la agenda de una barbería, se aplicaría la Ley N.º 29733 (Congreso de la República del Perú, 2011) y su reglamento (Ministerio de Justicia y Derechos Humanos, 2024): consentimiento previo e informado, uso solo para la finalidad declarada, derechos ARCO y medidas de seguridad sobre la base de datos.

6. **Privacidad entre hilos.** En el modelo, cada hilo guarda sus datos en su propia pila y ningún otro hilo los lee. Los datos compartidos solo se modifican dentro de la sección crítica.

8. Conclusiones

1. El problema del barbero dormilón se resuelve con dos semáforos contadores (`clientes` y `barberos`) y un mutex, y la misma solución funciona para cualquier número de barberos, sillas y clientes sin cambiar su estructura.

2. La solución está libre de interbloqueo porque ningún hilo retiene el mutex mientras espera otro semáforo, lo que rompe la condición de retención y espera.

3. Bloquear a los barberos en un semáforo, en lugar de usar espera activa, permite que duerman sin consumir CPU, como se observó en el caso de barberos ociosos con un uso del 22.7 %.

4. La exclusión mutua es indispensable: sin mutex, el simulador registró 7 inconsistencias del contador compartido y la barbería admitió más clientes de los que cabían.

5. La política de atención es una decisión de planificación. SJF y prioridad estática pueden producir inanición (esperas máximas de 63 u y 51 u), y el envejecimiento la reduce de forma efectiva (de 51 u a 32 u).

6. El simulador convierte un problema abstracto en algo observable: muestra los hilos, los semáforos, la memoria compartida y el efecto de cada decisión de diseño, y sirve como apoyo para la exposición en clase.

7. El software respeta la privacidad del usuario por diseño: procesa todo localmente, no recolecta datos personales y declara cómo se aplicaría la Ley N.º 29733 si llegara a manejar datos reales.

Referencias

Congreso de la República del Perú. (2011, 3 de julio). *Ley N.º 29733, Ley de Protección de Datos Personales*. Diario Oficial El Peruano. https://www.congreso.gob.pe/Docs/DGP/DIDP/files/ley_29733.pdf

Hoare, C. A. R. (1974). Monitors: An operating system structuring concept. *Communications of the ACM, 17*(10), 549–557. https://doi.org/10.1145/355620.361161

Hoare, C. A. R. (1978). Communicating sequential processes. *Communications of the*

ACM, 21(8), 666–677. https://doi.org/10.1145/359576.359585

Mandl, P. (2020). *Grundkurs Betriebssysteme: Architekturen, Betriebsmittelverwaltung, Synchronisation, Prozesskommunikation, Virtualisierung* (5.ª ed.). Springer Vieweg. https://doi.org/10.1007/978-3-658-30547-5

Maziero, C. A. (2019). *Sistemas operacionais: Conceitos e mecanismos*. Editora da UFPR. https://wiki.inf.ufpr.br/maziero/doku.php?id=socm:start

Ministerio de Justicia y Derechos Humanos. (2024). *Decreto Supremo N.º 016-2024-JUS, que aprueba el Reglamento de la Ley N.º 29733, Ley de Protección de Datos Personales*. Plataforma digital única del Estado peruano. https://www.gob.pe/institucion/anpd/normas-legales/6554453-16-2024-jus

Ruelas, D. (2026). *Sesión 05: Threads* [Diapositivas de clase]. Curso SIS 215 Sistemas Operativos, Universidad Nacional del Altiplano.

Wolf, G., Ruiz, E., Bergero, F., & Meza, E. (2015). *Fundamentos de sistemas operativos*. Universidad Nacional Autónoma de México, Instituto de Investigaciones Económicas y Facultad de Ingeniería. https://sistop.org/pdf/sistemas_operativos.pdf

Anexo A. Repositorio y ejecución

El código fuente del simulador se encuentra en el siguiente repositorio de GitHub:

https://github.com/Enerdev/Proyecto-final-Sistem-operative

- **Simulador:** abrir `simulador_barbero_dormilon.html` con cualquier navegador moderno (Chrome, Firefox, Edge). No requiere instalación ni conexión a internet; con conexión se cargan las tipografías.
- **Programa en C:** `gcc barbero.c -o barbero -pthread && ./barbero`
- **Programa en Python:** `python3 barbero.py`

Anexo B. Código fuente completo

Listado 3

Programa completo en C con `pthreads`

```c
/* Problema del Barbero Dormilón con N barberos - C + pthreads
 * Semáforos: clientes (clientes esperando), barberos (barberos listos)
 * Mutex: protege la variable compartida 'esperando'                    */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BARBEROS 2      /* número de hilos barbero          */
#define SILLAS   4      /* sillas de la sala de espera      */
#define CLIENTES 15     /* número de hilos cliente          */

sem_t clientes;                                 /* inicia en 0 */
sem_t barberos;                                 /* inicia en 0 */
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
int esperando = 0;                              /* memoria compartida */
int atendidos = 0, rechazados = 0;

void *barbero(void *arg) {
    int id = *(int *)arg;                       /* dato privado (pila) */
    while (1) {
        sem_wait(&clientes);                    /* duerme si no hay
            clientes */
        pthread_mutex_lock(&mutex);             /* --- sección crítica
            --- */
        esperando--;
        sem_post(&barberos);                    /* llama a un cliente */
        printf("[B%d] corta el pelo   | esperando = %d\n", id, esperando);
        pthread_mutex_unlock(&mutex);           /* --- fin sección crí
            tica --- */
        usleep((200 + rand() % 300) * 1000);    /* cortar_pelo() */
    }
    return NULL;
```

```c
}

void *cliente(void *arg) {
    int id = *(int *)arg;
    pthread_mutex_lock(&mutex);                    /* --- sección crítica --- */
    if (esperando < SILLAS) {
        esperando++;
        printf("  C%-2d se sienta       | esperando = %d\n", id, esperando);
        sem_post(&clientes);                       /* despierta a un barbero */
        pthread_mutex_unlock(&mutex);              /* liberar ANTES de esperar */
        sem_wait(&barberos);                       /* espera su turno */
        pthread_mutex_lock(&mutex); atendidos++; pthread_mutex_unlock(&mutex);
    } else {
        rechazados++;
        printf("  C%-2d se va: sala llena\n", id);
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main(void) {
    pthread_t tb[BARBEROS], tc[CLIENTES];
    int idb[BARBEROS], idc[CLIENTES];
    srand(7);
    sem_init(&clientes, 0, 0);
    sem_init(&barberos, 0, 0);

    for (int i = 0; i < BARBEROS; i++) {
        idb[i] = i + 1;
        pthread_create(&tb[i], NULL, barbero, &idb[i]);
    }
    for (int i = 0; i < CLIENTES; i++) {
        idc[i] = i + 1;
        pthread_create(&tc[i], NULL, cliente, &idc[i]);
        usleep((rand() % 250) * 1000);             /* llegada aleatoria */
    }
    for (int i = 0; i < CLIENTES; i++) pthread_join(tc[i], NULL);
    sleep(1);                                      /* deja terminar el último corte */

    printf("\nAtendidos: %d | Se fueron: %d\n", atendidos, rechazados);
    sem_destroy(&clientes);
    sem_destroy(&barberos);
    return 0;                                      /* termina el proceso y sus hilos */
}
```

Listado 4

Programa completo en Python con ***threading***

```python
# Problema del Barbero Dormilón con N barberos - Python + threading
import threading, time, random

BARBEROS, SILLAS, CLIENTES = 2, 4, 15
```

```
5
6  clientes = threading.Semaphore(0)    # clientes esperando
7  barberos = threading.Semaphore(0)    # barberos listos
8  mutex = threading.Lock()             # protege 'esperando'
9  esperando = 0                        # memoria compartida
10 atendidos = rechazados = 0
11 cerrar = threading.Event()
12
13 def barbero(id):
14     global esperando
15     while True:
16         clientes.acquire()                      # duerme si no hay clientes
17         if cerrar.is_set():
18             break
19         with mutex:                             # sección crítica
20             esperando -= 1
21             barberos.release()                  # llama a un cliente
22             print(f"[B{id}] corta el pelo    | esperando = {esperando}")
23         time.sleep(random.uniform(0.2, 0.5))    # cortar_pelo()
24     print(f"[B{id}] cierra su silla")
25
26 def cliente(id):
27     global esperando, atendidos, rechazados
28     with mutex:                                 # sección crítica
29         if esperando >= SILLAS:
30             rechazados += 1
31             print(f"  C{id:<2} se va: sala llena")
32             return
33         esperando += 1
34         print(f"  C{id:<2} se sienta       | esperando = {esperando}")
35         clientes.release()                      # despierta a un barbero
36     barberos.acquire()                          # espera su turno (fuera del
           mutex)
37     with mutex:
38         atendidos += 1
39
40 if __name__ == "__main__":
41     random.seed(7)
42     hb = [threading.Thread(target=barbero, args=(i + 1,)) for i in range(
           BARBEROS)]
43     for h in hb: h.start()
44     hc = []
45     for i in range(CLIENTES):
46         h = threading.Thread(target=cliente, args=(i + 1,))
47         h.start(); hc.append(h)
48         time.sleep(random.uniform(0, 0.25))     # llegada aleatoria
49     for h in hc: h.join()
50     time.sleep(0.6)                             # último corte
51     cerrar.set()
52     for _ in hb: clientes.release()             # despierta para cerrar
53     for h in hb: h.join()
54     print(f"\nAtendidos: {atendidos} | Se fueron: {rechazados}")
```